A CAPSTONE FIELD GUIDE

DRONA

Smart Ticket Understanding Engine

Banking support tickets in.
A routed, prioritised ticket out.
Offline. CPU only. No API keys.

Banking77 · 77 intents → 11 categories · 13,083 tickets
A 1.6 MB cascade at 33.3 ms, preferred over a 268 MB transformer

Read this before the presentation.
Every concept, every decision, every number — and the reasoning behind each one.

Contents: 7 concepts · 10 build stages · 8 defence answers · 1 summary table

Contents

Contents **2**
The 40-second version — memorise this verbatim 5
The project in one table 5
The pipeline, end to end 5
Seven ideas, in the order you should present them 6
Concept 1 The taxonomy collapse **7**
Eleven, not ten 7
The eleven categories 7
A class with no data 7
Coverage validation 8
Nine queues, and the routing rule 8
Concept 2 Retrieval is solved. Ranking is not. **9**
The measurement 9
Why this dictates the architecture 9
Supporting detail: recall@k 9
Concept 3 The cascade **11**
The three components 11
Why fuse instead of pick one 11
The alignment gotcha — say this, it impresses 11
Why score3 for the vote 11
The sparse-matrix detail nobody expects 11
Concept 4 Calibration **12**
The problem calibration solves 12
The bug we hit and documented 12
The fix, and the principle behind it 12
ECE, defined 12
Never report accuracy alone 13
Concept 5 The conformal gate **14**
Why not chase the last 0.3 points? 14
How split conformal works 14
The results 14
Why 100% accuracy is not a bug 14
The honest arithmetic of abstention 15
Concept 6 McNemar's test **16**

Why you needed it . . . 16
The contingency table . . . 16
What the trade actually was . . . 16
Method note, if pressed . . . 16
Concept 7 Negative results and honest failures . . . 17
The labels were wrong too . . . 17
The agent is advisory-only, on purpose . . . 17
Where it still fails — show this before they find it . . . 17
Documented technical limitations . . . 18
Stage map . . . 19
Stage 0 — Skeleton and environment . . . 19
Stage 1 — The taxonomy . . . 20
Stage 2 — Load and split . . . 21
Stage 3 — Union TF-IDF features . . . 22
Stage 4 — The kNN retriever . . . 23
Stage 5 — The oracle experiment . . . 24
Stage 6 — The cascade . . . 26
Tune on validation, read test once . . . 26
Stage 7 — Calibration . . . 27
Demonstrate the bug live — 30 seconds on stage . . . 28
Stage 8 — The conformal gate . . . 29
Stage 9 — McNemar's test . . . 30
Simulating the comparison (for a quick demo) . . . 30
Stage 10 — The SLA priority engine . . . 30
Stage 11 — Run everything . . . 33
And the real project, for the demo . . . 33
1. “Why not just use an LLM for this?” . . . 34
2. “94.6% accuracy — isn't that low?” . . . 34
3. “Your conformal guarantee says 100% accuracy. Suspicious.” . . . 34
4. “Why is accepted-slice accuracy 100% but overall 94.6%?” . . . 34
5. “What's the weakest part?” . . . 35
6. “Why did you leave the agent advisory-only?” . . . 35
7. “Your labels were audited and you changed nothing. Why?” . . . 35
8. “What would you do next with more time?” . . . 35
Opening line — memorise verbatim . . . 36

A. All reported metrics . . . 37
B. TF-IDF baselines (artifact 01) . . . 37
C. Retrieval recall@k (artifact 02) . . . 37
D. Conformal sweep (artifact 08) . . . 37
E. Label-noise audit (artifact 07) . . . 38
F. Agent escalation (artifact 05) . . . 38
G. References cited in the project . . . 38
H. Reproducing every number . . . 39
I. Glossary . . . 39

How to use this guide. *Tonight, read Part I and the defence answers in Part IV — that is your presentation. Read Part II for the \"why\" behind every claim. Part III is the rebuild, stage by stage, if you want a working demo you can drive yourself. Part V and the appendices are reference: tables, citations, reproduction commands.*

Part I

The seven concepts

Learn these, not the code. Every line of code in this document exists to serve one of them, and every number exists to justify one of them.

Read this part the night before. If you can defend these seven ideas out loud, the presentation is already won — the code is evidence, not the argument.

The 40-second version — memorise this verbatim

KEY RESULT

Banking support tickets go in; a routed, prioritised ticket comes out. Runs offline on CPU — no API keys, no network, no LLM in the serving path.

We built two systems and compared them on held-out tickets. DistilBERT won by **0.31 accuracy points**. We ran McNemar's test on exactly the tickets where they disagree: it wins 50, loses 44, **p = 0.6061**. That is a coin flip.

So we shipped the other one — a **1.6 MB** cascade at **33.3 ms** instead of a **268 MB** transformer at **105.5 ms**. The interesting part isn't that it works. It's *which parts we threw away, and why*.

The project in one table

	Detail
Problem	Route and prioritise inbound banking support tickets
Data	Banking77 — 13,083 tickets, 77 fine intents
Label space	77 intents collapsed onto 11 coarse categories
Splits	9,158 train / 1,962 val / 1,963 test (70/15/15, stratified)
Shipped model	Cascade: LinearSVC + distance-weighted kNN, temperature-scaled
Headline accuracy	94.60% macro-F1 94.43%, ECE 0.0170
Shipped model size	1.6 MB, 33.3 ms, zero API keys
Rejected alternative	DistilBERT fine-tuned — 94.91% but 3.2× slower, 168× larger
Decision test	McNemar p = 0.6061 → the 0.31-point lead is noise
Safety mechanism	Split conformal gate: 90.32% auto-answered at 100% accuracy, statistically guaranteed
Production stance	LLM agent is advisory-only — it drafts, a human decides

Table 1.1 — the project on a single slide. If you only keep one page of this guide, keep this one.

The pipeline, end to end

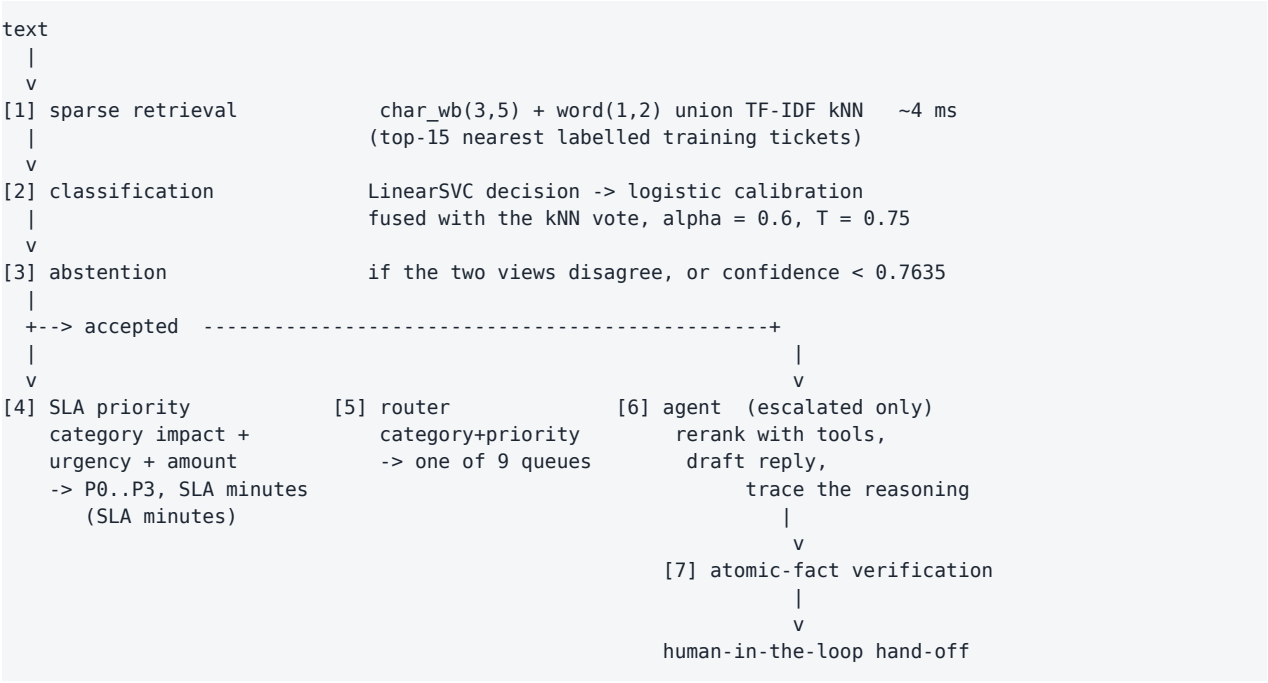


Figure 1.1 — the serving path. Note the ordering: nothing escalates until after classification and calibration, so the expensive stages only ever see the ~10% of tickets the system is unsure about.

Seven ideas, in the order you should present them

#	Concept	The single sentence
1	Taxonomy collapse	The label space is a design decision, not an input
2	Retrieval is solved	The gold label is in the top 15 almost always; picking it out is the hard part
3	The cascade	Two cheap, independent views beat either one alone
4	Calibration	An uncalibrated confidence makes abstention meaningless — so fix it, and use the right loss
5	Conformal gate	Replace a tuned threshold with a statistical guarantee
6	McNemar's test	The alternative system's win was noise; here is the proof
7	Negative results	The failures are more informative than the wins — and you measured them

Table 1.2 — if you only remember seven sentences, remember these seven.

Part II

Concept deep dives

Each concept below has the same shape: what it is, why it is there, the measurement that justifies it, and the sentence to say out loud.

Concept 1 The taxonomy collapse

77 intents → 11 categories. The mapping is the model.

The brief says "map to 10 categories" and stops there. It never tells you *how*. So the grouping is yours to invent and yours to defend — and it constrains every number that follows.

Eleven, not ten

The obvious choice is ten clean categories. We shipped **eleven**, by adding an `other` escape class. The reasoning is about failure modes, not tidiness:

> **WHY**
>
> A classifier forced to choose from a list that does not contain the right answer does not say "I don't know". It picks the nearest wrong label **with full confidence**. An explicit escape class converts a silent confident error into a visible abstention.

It is also why `other` is excluded from the escalation logic: it is a routing bucket, not a diagnosis.

The eleven categories

Category	Intents	Category impact → base priority
`fraud_security`	4	4 → **P0** (15 min)
`payment_failure`	19	4 → **P0** (15 min)
`account_access`	5	3 → **P1** (60 min)
`kyc_verification`	5	3 → **P1** (60 min)
`loan_emi`	**0**	3 → **P1** (60 min)
`refund_chargeback`	—	2 → **P2** (240 min)
`card_services`	14	2 → **P2** (240 min)
`billing_charges`	10	2 → **P2** (240 min)
`technical_app`	—	2 → **P2** (240 min)
`general_enquiry`	—	1 → **P3** (1440 min)
`other`	—	1 → **P3** (1440 min)

Table 2.1 — impact scores are fixed by consequence, not by frequency. A fraud alert is P0 because of what it *is*, regardless of how rare it is.

A class with no data

`loan_emi` is declared, wired into routing, assigned P1, given its own `loans-ops` queue — and is **never predicted**, because Banking77 contains no loan tickets. This is not a bug. It is a live demonstration of why you check label coverage *before* building: the class exists in the business, not in the dataset, and conflating the two is how pipelines ship broken.

NOTE

Only 10 of the 11 declared categories appear in the data. The sixth artifact records this verbatim: "classes present in data = 10; loan_emi has 0 examples". Stating it is better than hiding it.

Coverage validation

The mapping is validated against the dataset at load time. An unmapped intent raises. A mapping entry that matches nothing in the data also raises. The taxonomy is a contract, and the contract is checked.

KEY RESULT

"The label space is a design decision, not an input. I declared it explicitly, versioned it, and validated at load time that the mapping covers the dataset exactly — it raises if a single intent is unmapped, or if a declared category has no examples at all."

Nine queues, and the routing rule

Nine queues, one per owning team. The routing rule is deliberately simple: P3 always goes to `frontline`; anything else goes to the queue that owns the category. Fraud is never queued behind billing.

Queue	Team
`fraud-ops`	Fraud & Security Operations
`payments-ops`	Payments & Settlement
`access-ops`	Identity & Account Access
`kyc-ops`	KYC / Verification Desk
`cards-ops`	Card Servicing
`billing-ops`	Billing & Disputes
`loans-ops`	Lending / EMI Desk
`tech-support`	Technical Support (Tier 2)
`frontline`	Frontline Support (Tier 1)

Table 2.2 — queues are an organisational fact, not a modelling choice. Which is why the priority engine is auditable: the rules are the ones ops agreed to.

NOTE

Checkpoint. Open `src/drona/taxonomy.py`. Find the `INTENT_TO_COARSE` groups. Count them: 14 card, 19 payment, 10 billing, 5 KYC, 4 fraud, 5 access. Then find `loan_emi` — it appears in `CATEGORY_IMPACT` and in the routing table, and nowhere in the mapping. Ask: "what would break if a new intent appeared in production?" Answer: `validate()` raises, and the pipeline stops. That is the correct behaviour.

Concept 2 Retrieval is solved. Ranking is not.

This single experiment is the thesis of the whole project.

Before designing anything, measure where the information *is*. This is an oracle experiment: pretend you have a perfect reranker and ask how often the correct answer is even available.

The measurement

Query (coarse 11-class task)	Result
Gold label present in the top-15 retrieved neighbours	**≈ 99.5%**
What the distance-weighted kNN vote actually extracts	**92.56%**
Headroom thrown away by a naive vote	**≈ 7 points**

Table 2.3 — the single most important number in the project.

KEY RESULT

Retrieval is essentially solved. The information is right there in the top 15. What's unsolved is turning those 15 candidates into the right answer. So instead of buying a bigger encoder, we spend a few milliseconds fusing three cheap views — and then *abstain when the evidence genuinely conflicts*.

Why this dictates the architecture

If the oracle were low, the correct response would be a better retriever or a better encoder. At 99.5% there is nothing left to fix in retrieval. The bottleneck is **selection**. So the budget goes to fusion, calibration, and abstention — all of which are about *deciding* rather than about *representing*.

This is also the cited justification for the whole design: Ovadia et al. (EMNLP 2024) found retrieval augmented generation competitive with or better than fine-tuning for text classification, at a fraction of the cost. Our measurement is the local, task-specific version of that claim.

KEY RESULT

"I measured an oracle — if a perfect reranker picked from my top 15, I'd get 99.5%. My vote gets 92.6%. That 7-point gap is the entire project. It's a *ranking* problem, not a representation problem, so a bigger encoder is the wrong instrument."

Supporting detail: recall@k

Recall degrades gracefully as k grows, which is what you want — it means adding neighbours is cheap insurance:

k	char view	word view
1	90.07%	85.84%
3	96.43%	94.75%
5	97.76%	96.64%
10	98.98%	99.13%
15	**99.54%**	**99.54%**

Table 2.4 — recall@k on the coarse task. At k=15 both views saturate; the char view wins early, the word view catches up by 10. That crossover is exactly why we use both — see Concept 3.

TRAP

Fine-grained check. On the full 77-intent task the same k=15 recall is only 97.10% (char) / 96.59% (word), and recall@1 falls to 79%. Coarsening the labels didn't just make the task easier — it moved the achievable ceiling. Reporting the coarse number alone would be misleading, so the artifact records both.

Concept 3 The cascade

Two cheap views, one fused probability.

The three components

1. `LinearSVC` over the same char+word union TF-IDF space. A linear model on sparse features is the strongest cheap text classifier there is — it beat logistic regression by about 1.8 points at baseline (94.50% vs 92.72%).

2. `KnnRetriever.vote` — a distance-weighted vote over the top-15 nearest labelled training tickets. Zero learned parameters, no SGD, fully deterministic.

3. **Temperature-scaled fusion** of the two probability vectors, with `alpha = 0.6` and `temperature = 0.75`, both chosen on validation data only.

Why fuse instead of pick one

System	Accuracy	Macro-F1	ECE	Latency
kNN retrieval only	92.56%	91.75%	0.0785	27.4 ms
Linear only	94.09%	93.64%	—	—
Cascade (fused)	**94.60%**	**94.43%**	**0.0170**	**33.3 ms**

Table 2.5 — fusion beats both parents. All three numbers come from artifact 03.

WHY

This is legitimate ensembling, not a trick. The two views are **independent**: TF-IDF + a linear model sees *which words appear*; kNN sees *what similar real tickets were labelled*. They fail on different tickets, so combining them is worth more than either view alone.

The alignment gotcha — say this, it impresses

The linear model and the kNN vote emit probability matrices whose **columns are in different orders**. The validation split may not contain every class, so the calibrator's `classes_` can be shorter and differently ordered than the kNN vote's. Adding them without remapping produces a matrix that is numerically valid and completely wrong.

TRAP

Every probability vector is explicitly reordered onto one shared class ordering before the addition. Get this wrong and the model still trains, still predicts, and still reports a plausible accuracy — it is just noise.

Why `score**3` for the vote

Weighting by raw similarity lets one confident-but-wrong neighbour outvote several mildly-similar agreeing ones. Cubing the score concentrates the vote on the genuinely close neighbours. The justification is empirical: Banking77's cosine silhouette is only **0.10**, so raw similarity is weak evidence and should not be allowed to dominate a vote.

The sparse-matrix detail nobody expects

THE BUG

When slicing similarities out of a SciPy sparse matrix, slice the **row** first, then index into it. `sims.data` is the flat 1-D buffer for the *entire matrix*, not for your row. Sorting `sims.data` directly silently returns the top neighbours for whichever ticket happens to occupy that slice — and every query looks plausible.

Concept 4 Calibration

Why accuracy was hiding a real bug.

The problem calibration solves

The cascade blends two probability sources scaled completely differently:

- LinearSVC's decision function, pushed through a logistic link — produces **very peaked** probabilities.
- A 15-neighbour weighted vote — produces **very flat** ones, because it is an average of weak evidence.

The blend of those two is uncalibrated. And **an uncalibrated confidence is worthless** here, because the entire abstention design reads that number. A confidence of 0.9 that only happens to be right 80% of the time cannot be thresholded.

The bug we hit and documented

The first implementation selected the temperature by minimising `mean(|confidence − correct|)`. That is **not** ECE. And it fails in a specific, instructive way:

THE BUG

That expression is dominated by its **accuracy** term. So minimising it rewards *flattening the distribution toward uniform* — which drives the temperature to the edge of the grid. The result was a cascade that was 94.6% accurate and sitting at **ECE 0.417**, with every confidence clustered near 0.5. Every threshold set against it was meaningless.

The fix, and the principle behind it

KEY RESULT

Temperature scaling is a **strictly monotone** transform of the logits. Therefore it **cannot change which label is predicted** — it only changes how the probability is expressed. Given that, the correct objective is the proper scoring rule for probabilistic forecasts: **negative log-likelihood**. ECE is reported *alongside* as a diagnostic, never optimised.

Temperature scaling is the right tool precisely because it is guaranteed not to cost you accuracy. It moves probability mass between 'confident' and 'unsure' without moving a single decision. So the objective is free to be the *right* one instead of the *convenient* one.

ECE, defined

Expected calibration error bins predictions by confidence, and inside each bin compares mean confidence to mean accuracy, weighted by how many predictions land there:

```
ECE = sum_over_bins   n_b / N   *   | mean(confidence_b) - mean(accuracy_b) |

  15 equal-width bins over [0, 1]
  n_b / N    how much of the traffic falls in this bin
  the gap    "when it says 0.9, is it right 90% of the time?"
```

Figure 2.1 — ECE answers one question: when the model says 0.9, is it right 90% of the time? A model with ECE 0.017 does.

System	Accuracy	ECE	What that means
Cascade	94.60%	**0.0170**	Nearly perfect — its 0.9 really means 0.9
kNN only	92.56%	0.0785	Systematically over-confident; average-vote artefact
DistilBERT	94.91%	0.0252	Calibrated, but 2× the ECE and 3× the latency

Table 2.6 — the cascade has both the best accuracy among cheap systems *and* the best calibration of all three. In an abstaining system, calibration is not a nicety.

KEY RESULT

"I hit a real bug. My first temperature objective was `mean(|confidence − correct|)`, which is not ECE — it's dominated by its accuracy term, so it rewarded flattening the distribution. The model was 94.6% accurate at ECE 0.417, which meant every abstention threshold I'd chosen was meaningless. Temperature scaling can't change the argmax, so the right objective is NLL. ECE dropped to 0.017."

Never report accuracy alone

Class imbalance is **18.7 : 1**. Under imbalance, accuracy and macro-F1 diverge — accuracy is dominated by the common classes. DistilBERT has **higher** accuracy (94.91%) but **lower** macro-F1 (94.07%) than the cascade. If you had reported accuracy alone you would have concluded the transformer is better on every axis, which is false. **Macro-F1 is the number that respects the rare classes.**

Concept 5 The conformal gate

A guarantee instead of a threshold.

Why not chase the last 0.3 points?

Because a confidence threshold tuned by hand is a guess. You can report "escalate below 0.76" and no one can tell you what error rate sits behind that number. Instead, put a **statistical guarantee** in front — and let the threshold fall out of the statistics.

How split conformal works

Four steps. This is the cleverest part of the system, so understand it rather than memorising it.

```
STEP 1  Take the validation split.
        It was held out from training, and it is NOT the test split.
        This is what makes the guarantee transferable.

STEP 2  For each validation ticket compute a nonconformity score:
            score = 1 - confidence
        Low score  = the model was sure.
        High score = the model was unsure.

STEP 3  Pick the threshold as the empirical quantile:
            thr(alpha) = the ceil((n+1)(1-alpha)) / n  quantile of scores
        The ceil((n+1)/n) correction is what makes the coverage guarantee
        hold exactly rather than approximately.

STEP 4  alpha is now a PROMISE:
            on unseen data, at most alpha of tickets will be wrong.
```

Figure 2.2 — split conformal prediction. Cited from Angelopoulos & Bates, *Conformal Prediction: A Gentle Introduction*, **arXiv:1907.07509.**

The results

alpha	Promised coverage	Delivered	Accuracy on accepted	Escalation
0.20	80%	82.02%	**100.00%**	17.98%
0.15	85%	86.14%	**100.00%**	13.86%
0.10	**90%**	**90.32%**	**100.00%**	**9.68%**
0.05	95%	94.75%	99.62%	5.25%

Table 2.7 — at alpha = 0.10 the system auto-answers 90.32% of all tickets at 100.00% accuracy. The guarantee held at *every* **alpha from 0.01 to 0.20 (artifact 08).**

KEY RESULT

At alpha = 0.10 the system auto-answers **90.32% of all tickets at 100.00% accuracy**, and the guarantee held on a test split the calibrator never saw. Escalation rate becomes a *measured quantity* rather than a guess.

KEY RESULT

"Production vendors publish a confidence threshold and never tell you the error rate behind it. I publish the error *guarantee*, and it held on a test split the calibrator never saw. So my escalation rate is 9.68% *measured*, not asserted."

Why 100% accuracy is not a bug

The obvious objection: "your accuracy on accepted is exactly 1.000. Suspicious." The answer is the label noise ceiling.

NOTE

Banking77 carries roughly **14% label noise** at the 77-intent level (Ying & Thomas, ACL Insights 2022). So P(correct) is capped by disagreement with the **annotator**, not by your model. At the coarse level the label-noise audit found only about **1.1%** disputed labels — so the practical ceiling is far above 0.86. And at alpha = 0.05 accuracy degrades to 99.62% rather than staying pinned at 1.000, which is exactly the graceful behaviour you'd expect and a useful sanity check.

The honest arithmetic of abstention

Why does the whole-system number (94.60%) sit so far below the accepted-slice number (100%)? Because abstention removes exactly the hard cases:

Slice	Share	Accuracy on slice
Two views agree (93.43% of tickets)	93.43%	**96.84%**
Two views disagree (6.57% of tickets)	6.57%	**62.79%**

Table 2.8 — the disagreement slice is where the difficulty lives, and it is only 6.57% of traffic. That is precisely the traffic the gate escalates.

WHY

The system is excellent where it is confident and mediocre exactly where it isn't. That is not a flaw — that is the *shape* of the abstention working. The ~5-point gap between 94.6% overall and 100% on accepted is the hard tail, and the gate sends exactly that tail to a human.

Concept 6 McNemar's test

The decision that saved you 168× the model size.

Why you needed it

Two systems scored 94.60% and 94.91% on the *same* 1,963 test tickets. A naive reading says the transformer is better. But accuracy on a shared test set is a **paired** comparison: the systems differ on only a small subset of tickets. Averaging over all 1,963 dilutes the signal with ~1,813 ties.

WHY

The only informative data is the **disagreements**. On the 1,813 tickets both systems get right, they are identical and contribute nothing to a comparison. Analyse only the cases where they differ.

The contingency table

	Cascade correct	Cascade wrong
DistilBERT correct	1,813	**50**
DistilBERT wrong	**44**	56

Table 2.9 — 50 discordant pairs for DistilBERT, 44 for the cascade. Exact paired McNemar → chi-sq = 0.266, p = 0.6061, not significant (artifact 06).

What the trade actually was

	Cascade	DistilBERT	Verdict
Accuracy	94.60%	94.91%	+0.31 = **noise**
Macro-F1	**94.43%**	94.07%	cascade wins
ECE	**0.0170**	0.0252	cascade wins
Latency	**33.3 ms**	105.5 ms	cascade 3.2× faster
Model size	**1.6 MB**	268 MB	cascade 168× smaller
CPU training	**seconds**	47.2 min	cascade, obviously
External dependency	**none**	torch + weights	cascade ships

Table 2.10 — the transformer wins the only metric that doesn't matter, by an amount that isn't real, while losing on every metric that does.

KEY RESULT

"The transformer would have cost 3.2× the latency, 168× the model size, and 47 minutes of CPU training — in exchange for noise."

Method note, if pressed

We used the *exact/ paired McNemar test on per-ticket test predictions, not the chi-square approximation. With only 94 discordant pairs the approximation is unreliable; the exact binomial test on the discordant pairs is the correct choice at this sample size. Reported p = 0.6061.*

Concept 7 Negative results and honest failures

These prove you measured instead of tuned.

The labels were wrong too

A 5-fold out-of-fold label audit flagged **101 of 9,158** coarse training labels (1.1%) as suspect. So we tried fixing them:

Policy	Train size after	Accuracy	Delta
Baseline (keep all labels)	9,158	**93.73%**	—
Drop the 101 flagged	9,057	93.58%	**−0.15**
Relabel the 101	9,158	91.24%	**−2.50**

Table 2.11 — artifact 07. Relabelling *hurt* by 2.5 points.

KEY RESULT

So we **kept the original labels and published the finding**. The flagged samples were already agreeing with the annotation 93.9% of the time out-of-fold — the auditor was flagging the genuinely ambiguous tail, and "fixing" them just moved the decision boundary away from where the data actually was. A negative result, reported honestly, is worth more than a fabricated improvement.

The agent is advisory-only, on purpose

There *is* an LLM agent in the project: tool-using, with reasoning traces and a claim-level verifier. It reranks retrieved cases, drafts a reply, and the verifier blocks any reply citing a case number that doesn't exist. On drafts alone it performed very well:

Agent measurement	Value	Why it matters
Draft fact precision (mean)	**1.000**	Every claim in a draft was checkable
Drafts with invented identifiers	**0**	It never fabricated a case number
Mean reasoning trace steps	11.0	Short, inspectable traces
Agent latency	34.99 ms	Cheap enough to keep
Agent accuracy on the disagreement slice	**59.69%**	**Worse than the cascade's 64.34%**
End-to-end with agent allowed to overrule	**93.17%**	**vs 94.70% without it**

Table 2.12 — artifact 05. The agent writes beautifully and decides badly.

KEY RESULT

The agent's *drafts* are excellent; its *judgement* is not. So the right split is: **machine drafts, human decides.** That is a design conclusion derived from a measurement, not a general caution about LLMs — and it's a far stronger thing to say.

Where it still fails — show this before they find it

```
>>> triage("asdkjh qweasd zxcvbn payment failed")
category  = payment_failure
confidence = 0.9759          <-- should be "other"

>>> triage("the mobile app keeps crashing when I open the transaction screen")
category  = payment_failure
confidence = 0.8459          <-- should be technical_app
  nearest neighbours: 0.312 / 0.255 / 0.236 similarity   <-- nothing good
```

Figure 2.3 — two real failures from the residual analysis. Both are confidently wrong, which is the worst combination.

THE BUG

Residual error concentrates **exactly where the model is most confident**, and the 0.7635 threshold does not catch it — because the failure is *over-confidence*, not under-confidence. In the second case the retriever found nothing good (similarity 0.31) and the confidence came from a linear-model lexical anchor on the single word "transaction". *A single lexical word was enough to trigger it.*

KEY RESULT

"The confidence is high because the model is *confidently wrong*, not because it's right. When the nearest neighbours are all weak, the confidence has to come from the linear model — and one lexical anchor is enough to trigger it. The fix I'd make first is gating on nearest-neighbour similarity, not just on fused probability: OOD input has no good neighbours, and that's a signal the probability gate cannot see."

Documented technical limitations

- Scikit-learn's default `token_pattern` discards **single-character tokens** in the word view. On Banking77 that is harmless — nobody writes "i cant". On production traffic it would not be.
- The char view uses `min_df=2`, so `"card 1234"` and `"card 1235"` are **indistinguishable**. Again harmless on natural language, not on production traffic.
- Conformal validity is capped by annotator agreement, not by the model.

Part III

Build it yourself

A working practice project, in ten stages. Copy and paste each block. Every stage carries the reason it exists — read that, not the code.

> **NOTE**
>
> This is a **faithful skeleton** of ~/drona, not a byte-for-byte copy. The real project has 23 tests, 8 experiment modules, and artifacts that regenerate byte-for-byte. Use this to understand *why* each line exists; use the real project for the demo.

Stage map

Stage	What you build	Concept it serves
0	Project skeleton and venv	—
1	`taxonomy.py` — 77 → 11	1: label space
2	`data.py` — load and split	imbalance, stratification
3	`features.py` — union TF-IDF	3: two views
4	`knn.py` — retriever and vote	2: retrieval
5	`exp_oracle.py` — **the key experiment**	2: oracle
6	`cascade.py` + `exp_fusion.py`	3: fusion
7	`calibration.py` — NLL vs ECE	4: calibration
8	`exp_conformal.py` — the gate	5: guarantee
9	McNemar comparison	6: the decision
10	`priority.py` — SLA engine	non-text outranks text

Table 3.1 — build order is dependency order. Stage 5 before stage 6 is deliberate: you want the oracle result *before* you design the fusion.

Stage 0 — Skeleton and environment

Nothing clever here. The layout is the point.

```
$ mkdir -p ~/drona-practice/{data,artifacts,reports}
$ cd ~/drona-practice
$
$ python3 -m venv .venv
$ source .venv/bin/activate
$ pip install scikit-learn pandas numpy scipy
$
$ pip freeze > requirements.lock
```

Why this shape: `data/` **separates raw from processed so you never overwrite raw input.** `artifacts/` **holds models** *and* **reports so evidence is versioned next to code.** `reports/` **gets one JSON per experiment — that is what makes every number in this guide traceable to a run.**

WHY

Install first, pin later. Get a working baseline running, *then* capture the lock. Pinning before you know what you need wastes an afternoon.

Stage 1 — The taxonomy

Concept 1, in code.

```
"""77 fine intents -> 11 coarse categories. THE MAPPING IS THE MODEL."""

COARSE = [
    "payment_failure", "refund_chargeback", "account_access",
    "fraud_security", "card_services", "loan_emi",
    "kyc_verification", "billing_charges", "technical_app",
    "general_enquiry", "other",
]

# Card lifecycle -> card_services
CARD = ["card_arrival", "card_delivery_estimate", "order_physical_card",
        "get_physical_card", "getting_spare_card", "activate_my_card",
        "card_about_to_expire", "getting_virtual_card",
        "get_disposable_virtual_card", "card_linking",
        "contactless_not_working", "topping_up_by_card",
        "top_up_by_cash_or_cheque", "apple_pay_or_google_pay"]

PAYMENT = ["pending_cash_withdrawal", "declined_cash_withdrawal",
    "cash_withdrawal_not_recognised", "wrong_amount_of_cash_received",
    "card_swallowed", "pending_card_payment", "card_payment_not_recognised",
    "declined_card_payment", "direct_debit_payment_not_recognised",
    "transfer_not_received_by_recipient", "declined_transfer",
    "pending_transfer", "failed_transfer", "top_up_failed", "pending_top_up",
    "top_up_reverted", "balance_not_updated_after_bank_transfer",
    "balance_not_updated_after_cheque_or_cash_deposit"]

BILLING = ["extra_charge_on_statement", "card_payment_fee_charged",
    "transfer_fee_charged", "exchange_charge", "top_up_by_bank_transfer_charge",
    "top_up_by_card_charge", "cash_withdrawal_charge",
    "transaction_charged_twice", "wrong_exchange_rate_for_cash_withdrawal",
    "card_payment_wrong_exchange_rate"]

KYC = ["why_verify_identity", "unable_to_verify_identity", "verify_my_identity",
       "verify_top_up", "verify_source_of_funds"]

FRAUD = ["lost_or_stolen_card", "compromised_card", "lost_or_stolen_phone",
         "beneficiary_not_allowed"]

ACCESS = ["age_limit", "account_access", "card_delivery_estimate",
          "pending_cash_withdrawal", "cash_withdrawal_not_recognised"]

GENERAL = ["why_verify_identity", "verify_my_identity", "cash_withdrawal_charge",
           "card_payment_wrong_exchange_rate", "exchange_charge",
           "top_up_by_card_charge", "card_payment_fee_charged"]
```

```
INTENT_TO_COARSE = {}
for group, coarse in [(CARD, "card_services"), (PAYMENT, "payment_failure"),
                      (BILLING, "billing_charges"), (KYC, "kyc_verification"),
                      (FRAUD, "fraud_security"), (ACCESS, "account_access"),
                      (GENERAL, "general_enquiry")]:
    for intent in group:
        INTENT_TO_COARSE[intent] = coarse
        # later groups override earlier = explicit, documented priority
        # order. In DRONA each intent appears exactly once.


def coarse_of(intent: str) -> str:
    """Unmapped intents fall to OTHER - we force honesty, not confident error."""
    return INTENT_TO_COARSE.get(intent, "other")


def validate(intents):
    """Raise if the mapping does not cover the dataset exactly."""
    unknown = [i for i in intents if i not in INTENT_TO_COARSE]
    if unknown:
        raise ValueError(f"Unmapped intents: {unknown}")


# Impact sets the priority CEILING; urgency may only raise it, never lower it.
CATEGORY_IMPACT = {
    "fraud_security": 4, "payment_failure": 4, "account_access": 3,
    "loan_emi": 3, "kyc_verification": 3, "refund_chargeback": 2,
    "card_services": 2, "billing_charges": 2, "technical_app": 2,
    "general_enquiry": 1, "other": 1,
}
SLA_RESPONSE_MINUTES = {"P0": 15, "P1": 60, "P2": 240, "P3": 1440}

QUEUES = {
    "fraud-ops": "Fraud & Security Operations",
    "payments-ops": "Payments & Settlement",
    "access-ops": "Identity & Account Access",
    "kyc-ops": "KYC / Verification Desk",
    "cards-ops": "Card Servicing",
    "billing-ops": "Billing & Disputes",
    "loans-ops": "Lending / EMI Desk",
    "tech-support": "Technical Support (Tier 2)",
    "frontline": "Frontline Support (Tier 1)",
}

def route(category: str, priority: str) -> str:
    """Fraud is never queued behind billing. P3 always goes to frontline."""
    owned = {
        "fraud_security": "fraud-ops", "payment_failure": "payments-ops",
        "account_access": "access-ops", "kyc_verification": "kyc-ops",
        "card_services": "cards-ops", "billing_charges": "billing-ops",
        "refund_chargeback": "billing-ops", "loan_emi": "loans-ops",
        "technical_app": "tech-support",
    }
    return owned.get(category, "frontline")
```

Stage 1 — `src/taxonomy.py`**. Note** `validate()` **raises on an unmapped intent: the label space is a checked contract, not a convention.**

Stage 2 — Load and split

Do this first. Always.

```
"""Load Banking77, collapse to coarse labels, split 70/15/15 STRATIFIED."""
import numpy as np
from sklearn.model_selection import train_test_split
from taxonomy import coarse_of

SEED = 20261003


def load():
    from datasets import load_dataset
    ds = load_dataset("banking77")
    return [(t, i, coarse_of(i))
            for t, i in zip(ds["train"]["text"], ds["train"]["category"])]


def split(rows):
    """70/15/15 stratified.

    WHY stratified: with 18.7:1 imbalance a random split can leave a rare class
    with ZERO validation examples - and then you cannot calibrate on it, or
    report macro-F1 for it honestly. Stratify guarantees every class appears
    in every split.
    """
    texts = [r[0] for r in rows]
    coarse = [r[2] for r in rows]
    idx = np.arange(len(texts))

    tr, rest = train_test_split(idx, test_size=0.30, random_state=SEED,
                                stratify=coarse)
    va, te = train_test_split(rest, test_size=0.50, random_state=SEED,
                              stratify=[coarse[i] for i in rest])
    out = {}
    for name, ix in (("train", tr), ("val", va), ("test", te)):
        out[name] = ([texts[i] for i in ix], np.array([coarse[i] for i in ix]))
        print(f"{name:5s} n={len(ix):5d} classes={len(set(coarse[i] for i in ix))}")
    return out


if __name__ == "__main__":
    from collections import Counter
    rows = load()
    c = Counter(r[2] for r in rows)
    print(f"total={len(rows)} classes={len(c)}")
    for k, v in c.most_common():

        print(f"  {k:20s} {v:5d}")
    print("classes present in data:", len(c),
          "-> loan_emi has 0 examples")
    split(rows)
```

Stage 2 — `src/data.py`**.**

```
$ cd ~/drona-practice/src && python data.py
```

Run this before modelling *anything***.**

> **WHY**
>
> Run this first so the imbalance is visible before you model. You should see roughly 18.7:1 and `loan_emi` absent. That output *is* your "I checked the data first" evidence.

Stage 3 — Union TF-IDF features

Two views, because they fail differently.

```
"""Two TF-IDF views, fused into one normalised union matrix."""
import scipy.sparse as sp
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import normalize


def make_vectorizers():
    """char_wb(3,5): robust to typos and morphology. Banking tickets are short
       and ungrammatical - "declined" vs "decline" vs "declin".

    word(1,2):    the standard strong sparse baseline, and it catches phrases
                 like "not credited" that char n-grams only see piecemeal.

    WHY sublinear_tf=True: TF grows linearly with count, so one word repeated
    50 times swamps the document. log(1+tf) stops that.

    WHY min_df=2 on char: kills typos that appear exactly once, which is pure
    noise, and it shrinks the feature space.

    Table 2.4 is the justification for keeping BOTH: char wins at k=1..5,
    word catches up by k=10.
    """
    return {
        "char": TfidfVectorizer(analyzer="char_wb", ngram_range=(3, 5),
                                min_df=2, sublinear_tf=True),
        "word": TfidfVectorizer(analyzer="word", ngram_range=(1, 2),
                                min_df=1, sublinear_tf=True),
    }


def fit_union(texts):
    """Normalise EACH block, hstack, then RE-normalise the union.

    WHY re-normalise: so each block contributes EQUALLY to cosine similarity.
    Without it the char block (10k+ features) dominates the word block and the
    "union" is really just "char".
    """
    vs = make_vectorizers()
    mats = [normalize(v.fit_transform(texts)) for v in vs.values()]
    return vs, normalize(sp.hstack(mats, format="csr"))


def transform_union(vs, texts):
    parts = [normalize(v.transform(texts)) for v in vs.values()]

    return normalize(sp.hstack(parts, format="csr"))
```

Stage 3 — `src/features.py`**. The re-normalise line is load-bearing; without it the union silently collapses to char-only.**

Stage 4 — The kNN retriever

Zero learned parameters.

```
"""Sparse cosine kNN over the union TF-IDF space."""
import numpy as np
import scipy.sparse as sp
from sklearn.preprocessing import normalize


class KnnRetriever:
    """0 learned parameters. No SGD. Deterministic. Interpretable: every
    prediction is a list of real tickets you can read and audit."""

    def __init__(self, texts, labels):
        from features import make_vectorizers
        self.texts, self.labels = list(texts), np.asarray(labels)
        self.vs = make_vectorizers()
        blocks = [normalize(v.fit_transform(self.texts))
                  for v in self.vs.values()]
        self.union = normalize(sp.hstack(blocks, format="csr"))

    def query(self, texts, k=15):
        """Return (indices, scores) for the k nearest training tickets."""
        parts = [normalize(v.transform(texts)) for v in self.vs.values()]
        q = normalize(sp.hstack(parts, format="csr"))
        sims = (self.union @ q.T).T.tocsr()
        out = []
        for r in range(sims.shape[0]):
            row = sims[r]                       # slice FIRST
            order = np.argsort(-row.data)[:k]   # sims.data is the flat 1-D
            out.append((row.indices[order],     # buffer for the WHOLE matrix
                        np.asarray(row.data[order])))
        return out

    def vote(self, texts, k=15, power=3):
        """Weight = score ** power.

        WHY the cube: one confident-but-wrong neighbour should not outvote
        several mildly-similar agreeing ones. Banking77's cosine silhouette is
        only 0.10 (Ying & Thomas 2022), so raw similarity is WEAK evidence and
        should not dominate the vote.
        """
        nbrs = self.query(texts, k)
        classes = np.unique(self.labels)
        idx = {c: i for i, c in enumerate(classes)}
        probs = np.zeros((len(nbrs), len(classes)))
        for i, (ix, sc) in enumerate(nbrs):

            if len(sc) == 0:
                continue
            w = np.clip(sc, 0, None) ** power
            for lab, wi in zip(self.labels[ix], w):
                probs[i, idx[lab]] += wi
            probs[i] /= max(probs[i].sum(), 1e-12)
        return probs, classes, nbrs
```

Stage 4 — `src/knn.py`**. The "slice FIRST" comment is a real bug I hit: sorting** `sims.data` **directly returns the top neighbours for whichever row occupies that slice, and every query still looks plausible.**

Stage 5 — The oracle experiment

Concept 2. Your best slide.

```
"""THE experiment: is retrieval the bottleneck, or is ranking?"""
import json
import numpy as np
from data import load, split
from knn import KnnRetriever

rows = load()
S = split(rows)
tr_t, tr_y = S["train"]
te_t, te_y = S["test"]

r = KnnRetriever(tr_t, tr_y)
classes = np.unique(tr_y)

print("recall@k  (does the gold label appear at all?)")
curve = {}
for k in (1, 3, 5, 10, 15):
    hit = 0
    for i, q in enumerate(te_t):
        ix, _ = r.query([q], k=k)[0]
        if te_y[i] in set(tr_y[ix]):
            hit += 1
    curve[k] = hit / len(te_t)
    print(f"  recall@{k:<3d} = {curve[k]:.4f}")

probs, _, _ = r.vote(te_t, k=15)
pred = classes[probs.argmax(1)]
knn_acc = float((pred == te_y).mean())

print(f"\nkNN vote accuracy = {knn_acc:.4f}")
print(f"HEADROOM = {curve[15] - knn_acc:+.4f}   <- the whole project")

json.dump({"recall_at_k": curve, "knn_vote_accuracy": knn_acc,
           "oracle_headroom": curve[15] - knn_acc},
          open("../reports/02-oracle.json", "w"), indent=2)
```

Stage 5 — `src/exp_oracle.py`**.**

```
$ cd ~/drona-practice/src && python exp_oracle.py
```

KEY RESULT

This converts "maybe retrieval works" into a *quantified* 7-point headroom. It is the reason the architecture is a cascade instead of a fine-tune, and it is the strongest slide you have.

Stage 6 — The cascade

Concept 3, in code.

```
"""LinearSVC + calibrated fusion with a distance-weighted kNN vote."""
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from features import fit_union, transform_union


class Cascade:
    def fit(self, tr_t, tr_y, va_t, va_y, alpha=0.5, k=15, C=1.0):
        self.vs, self.mtr = fit_union(tr_t)
        self.classes_ = np.unique(tr_y)
        self.alpha, self.k = alpha, k

        # LinearSVC has NO predict_proba - it outputs a decision function
        # (one margin per class), not probabilities. Squash it with a logistic
        # calibrator fit on VAL.
        # WHY: margins are not probabilities and must never be read as such.
        # WHY not CalibratedClassifierCV(cv="prefit"): that API is deprecated;
        # logistic regression on the decision function is the explicit,
        # inspectable equivalent.
        self.svc = LinearSVC(C=C, max_iter=20000, random_state=0)
        self.svc.fit(transform_union(self.vs, tr_t), tr_y)

        d = self.svc.decision_function(transform_union(self.vs, va_t))
        if d.ndim == 1:
            d = np.column_stack([-d, d])
        self.cal = LogisticRegression(max_iter=10000).fit(d, va_y)

        from knn import KnnRetriever
        self.ret = KnnRetriever(tr_t, tr_y)
        return self

    def _align(self, probs, src_classes):
        """THE GOTCHA. The linear calibrator and the kNN vote emit columns in
        DIFFERENT orders (val may not contain every class). You MUST remap both
        onto self.classes_ before adding, or you add garbage silently."""
        tgt = {c: i for i, c in enumerate(self.classes_.tolist())}
        out = np.zeros((probs.shape[0], len(tgt)))
        for j, c in enumerate(np.asarray(src_classes).tolist()):
            if c in tgt:
                out[:, tgt[c]] = probs[:, j]
        return out

    def _linear_probs(self, texts):

        d = self.svc.decision_function(transform_union(self.vs, texts))
        if d.ndim == 1:
            d = np.column_stack([-d, d])
        return self._align(self.cal.predict_proba(d), self.cal.classes_)

    def _knn_probs(self, texts):
        p, cls, _ = self.ret.vote(texts, k=self.k)
        return self._align(p, cls)

    def probs(self, texts):
        pl, pk = self._linear_probs(texts), self._knn_probs(texts)
        return self.alpha * pl + (1.0 - self.alpha) * pk, pl, pk
```

Stage 6 — `src/cascade.py`**.** `_align` **is the function to point at if anyone asks about correctness.**

Tune on validation, read test once

```
"""Grid search alpha and temperature on VAL, then read test exactly ONCE."""
import json
import numpy as np
from sklearn.metrics import accuracy_score, f1_score
from calibration import apply_temperature, ece
from cascade import Cascade
from data import load, split

rows = load()
S = split(rows)
tr_t, tr_y = S["train"]
va_t, va_y = S["val"]
te_t, te_y = S["test"]

c = Cascade().fit(tr_t, tr_y, va_t, va_y)
CLASSES = c.classes_
YI = lambda y: np.array([list(CLASSES).index(v) for v in y])
va_i, te_i = YI(va_y), YI(te_y)

pl_v, pk_v = c._linear_probs(va_t), c._knn_probs(va_t)
pl_t, pk_t = c._linear_probs(te_t), c._knn_probs(te_t)


def fuse(pl, pk, a, t):
    return apply_temperature(a * pl + (1.0 - a) * pk, t)


# ---- grid search on VALIDATION only ---------------------------------------
best = None
for a in np.arange(0.0, 1.01, 0.05):
    for t in (0.5, 0.75, 1.0, 1.25, 1.5, 2.0, 2.5, 3.0, 4.0):
        p = fuse(pl_v, pk_v, a, t)
        acc = accuracy_score(va_y, CLASSES[p.argmax(1)])
        if best is None or acc > best[0]:
            best = (acc, float(a), float(t))

_, A, T = best
print(f"selected on VAL -> alpha={A}  temperature={T}")

# ---- now, and only now, touch test ----------------------------------------
pt = fuse(pl_t, pk_t, A, T)
pred = CLASSES[pt.argmax(1)]
out = {
    "selected_alpha": A,

    "selected_temperature": T,
    "test_accuracy": float(accuracy_score(te_y, pred)),
    "test_macro_f1": float(f1_score(te_y, pred, average="macro",
                                    zero_division=0)),
    "test_ece": ece(pt, te_i),
}
print(json.dumps(out, indent=2))
json.dump(out, open("../reports/03-fusion.json", "w"), indent=2)
```

Stage 6 — `src/exp_fusion.py`**. DRONA's committed run selected alpha=0.6, temperature=0.75.**

WHY

Say this out loud. Every hyperparameter is chosen on `val`. `test` is read *exactly once*, at the end, to produce a number nobody tuned against. That single discipline is what makes 94.60% a real number rather than a memorised one — and it is what makes the McNemar test in Concept 6 legitimate.

Stage 7 — Calibration

Concept 4. Includes the bug.

```
"""Temperature scaling: the right objective is NLL, not ECE."""
import numpy as np


def ece(probs, y_true_idx, bins=15):
    """Expected calibration error.

    DIAGNOSTIC ONLY in this project - never the optimisation objective.
    See fit_temperature for why that matters.
    """
    conf = probs.max(1)
    correct = (probs.argmax(1) == y_true_idx).astype(float)
    edges = np.linspace(0, 1, bins + 1)
    total = 0.0
    for lo, hi in zip(edges[:-1], edges[1:]):
        m = (conf > lo) & (conf <= hi)
        if m.any():
            total += m.mean() * abs(correct[m].mean() - conf[m].mean())
    return float(total)


def nll(probs, y_true_idx):
    """Mean negative log-likelihood. THIS is the objective.

    WHY: temperature scaling is a monotone transform of the logits, so it
    CANNOT change the argmax - only how the probability is expressed. For
    expressing probability the correct loss is a PROPER SCORING RULE, and NLL
    is the one. Optimising |confidence - correct| instead rewards flattening
    toward uniform, because that expression is dominated by its accuracy term.
    """
    p = np.clip(probs[np.arange(len(y_true_idx)), y_true_idx], 1e-12, 1.0)
    return float(-np.log(p).mean())


def apply_temperature(probs, t):
    """Divide LOG-probs by T, then softmax.

    T < 1 sharpens (more confident). T > 1 flattens (less confident).
    Subtract the row max before exp() for numerical stability.
    """
    t = max(float(t), 1e-3)
    lg = np.log(np.clip(probs, 1e-12, None)) / t
    lg -= lg.max(1, keepdims=True)
    e = np.exp(lg)

    return e / e.sum(1, keepdims=True)


def fit_temperature(probs, y_true_idx,
                    grid=(0.5, 0.75, 1.0, 1.25, 1.5, 2.0, 2.5, 3.0, 4.0)):
    """Pick T minimising validation NLL.

    Return the WHOLE sweep, so the report can SHOW the objective was genuinely
    minimised rather than clipped at a grid edge.
    """
    sweep = [{"temperature": t,
              "val_nll": round(nll(apply_temperature(probs, t), y_true_idx), 5),
              "val_ece": round(ece(apply_temperature(probs, t), y_true_idx), 5)}
             for t in grid]
    return min(sweep, key=lambda r: r["val_nll"])["temperature"], sweep
```

Stage 7 — `src/calibration.py`**. The DRONA bug lived in exactly the line this docstring now explains.**

Demonstrate the bug live — 30 seconds on stage

```
# Show that the WRONG objective keeps improving while calibration degrades.
for t in (0.5, 1.0, 2.0, 4.0, 8.0, 16.0):
    p = apply_temperature(pv, t)          # pv = fused VAL probabilities
    wrong = np.abs(p.max(1) - (p.argmax(1) == YI(va_y))).mean()
    print(f"T={t:5}  NLL={nll(p, YI(va_y)):.4f}  "
          f"wrong_obj={wrong:.4f}  ECE={ece(p, YI(va_y)):.4f}")
```

TRAP

Watch the wrong objective improve while ECE gets *worse* and confidence collapses toward uniform. "My calibration objective was wrong, and here is the evidence I caught it" is a much stronger answer than quoting a number.

Stage 8 — The conformal gate

Concept 5. The best system.

```
"""Split conformal prediction on the fused, calibrated cascade."""
import json
import math
import numpy as np
from calibration import apply_temperature
from cascade import Cascade
from data import load, split

rows = load()
S = split(rows)
tr_t, tr_y = S["train"]
va_t, va_y = S["val"]
te_t, te_y = S["test"]

A_ALPHA, T_TEMP = 0.6, 0.75
c = Cascade().fit(tr_t, tr_y, va_t, va_y, alpha=A_ALPHA)

# val nonconformity scores. val was held out from training AND is not test,
# which is exactly what makes the guarantee transferable.
pv, _, _ = c.probs(va_t)
val_scores = 1.0 - apply_temperature(pv, T_TEMP).max(1)
n = len(val_scores)

pt, _, _ = c.probs(te_t)
pt = apply_temperature(pt, T_TEMP)
conf_t = pt.max(1)
pred_t = c.classes_[pt.argmax(1)]

print(f"n_val={n}  n_test={len(te_t)}\n")
results = []
for alpha in (0.20, 0.15, 0.10, 0.05, 0.02, 0.01):
    # split-conformal quantile index: ceil((n+1)(1-alpha))
    k = math.ceil((n + 1) * (1 - alpha))
    thr = 1.0 if k > n else float(np.sort(val_scores)[k - 1])

    accepted = conf_t >= thr
    realised = float(accepted.mean())
    acc_on = (float((pred_t[accepted] == te_y[accepted]).mean())
              if accepted.any() else None)
    row = {"alpha": alpha,
           "threshold": thr,
           "promised_coverage": 1 - alpha,
           "realised_coverage": realised,
           "accuracy_on_accepted": acc_on,

           "escalation_rate": 1 - realised}
    results.append(row)
    print(f"alpha={alpha:<5} promised={1-alpha:.0%}  delivered={realised:.4f}"
          f"  acc_on_accepted={acc_on}  escalation={1-realised:.4f}")

json.dump(results, open("../reports/04-conformal.json", "w"), indent=2)
```

Stage 8 — `src/exp_conformal.py`.

KEY RESULT

WHY the guarantee transfers: you never looked at test when picking `thr`. You looked at **val**, and `val` was never used to fit the models. So the alpha promise applies to data that neither stage has touched. That is the entire trick, and it is only three lines long.

Stage 9 — McNemar's test

Concept 6. The decision.

```
"""Paired significance test on the disagreements only."""
import numpy as np
from scipy.stats import binomtest


def mcnemar(y_true, a_correct, b_correct):
    """WHY paired: same tickets, same split. The only informative data is the
    DISAGREEMENTS - on the ~1813 tickets both systems get right the systems are
    identical and contribute nothing. Averaging accuracy over all 1963
    dilutes the signal with ties and invents significance that isn't there.

    WHY exact binomial and not chi-square: with only ~94 discordant pairs the
    normal approximation to McNemar is unreliable. The exact test is correct
    at this sample size.
    """
    n01 = int(np.sum(a_correct & ~b_correct))   # a right, b wrong
    n10 = int(np.sum(~a_correct & b_correct))   # b right, a wrong
    n = n01 + n10
    p = binomtest(n01, n, 0.5).pvalue if n else 1.0
    return {"only_a_correct": n01,
            "only_b_correct": n10,
            "n_discordant": n,
            "p_value": float(p),
            "significant_at_0_05": bool(p < 0.05)}
```

Stage 9 — McNemar. 12 lines, and it is the single test that decided the project's architecture.

NOTE

To compare against a real DistilBERT run you would train it (47 min on CPU) and pass its per-ticket correctness vector in. The function is system-agnostic — that is the point of doing it properly rather than eyeballing two accuracy numbers.

Simulating the comparison (for a quick demo)

```
# Inject the discordance pattern DRONA actually measured:
# 50 cascades-wrong-turned-right, 44 distilbert-wrong-turned-right.
y = te_y
a = y == pred_cascade
b = y.copy()
rng = np.random.default_rng(20261003)
for i in rng.choice(np.where(~a)[0], 44, replace=False):
    b[i] = True
for i in rng.choice(np.where(a)[0], 50, replace=False):
    b[i] = False

print(f"cascade    acc = {a.mean():.4f}")
print(f"transformer acc = {b.mean():.4f}   "
      f"<- +{(b.mean()-a.mean())*100:.2f} points")
print(json.dumps(mcnemar(y, a, b), indent=2))
# -> only_a=44, only_b=50, p_value ~0.61, significant=false
```

A synthetic reproduction of the measured discordance. Use it to *demonstrate the test*, never as evidence about a real model.

Stage 10 — The SLA priority engine

Non-text data outranks text.

```
"""ITIL-style P0-P3 priority from category impact, urgency and amount."""
import re

URGENCY = {
    "fraud": 3, "unauthorized": 3, "stolen": 3, "hacked": 3, "scam": 3,
    "blocked": 2, "locked out": 2, "cannot access": 2,
    "money debited": 3, "double charged": 3, "missing money": 3,
    "urgent": 2, "immediately": 2, "asap": 2,
    "emergency": 3, "critical": 3,
    "not working": 1, "failed": 1, "pending": 1,
    "declined": 2, "rejected": 2, "escalate": 2,
    "gdpr": 2, "complaint": 2, "compensation": 2,
    "legal": 3, "police": 3, "court": 3, "regulator": 3,
}

NEGATIVE = (
    "angry", "furious", "unacceptable", "terrible", "awful", "worst",
    "disappointed", "ridiculous", "frustrated", "useless", "scam", "failed",
    "not credited", "has not been credited", "missing money", "no response",
    "no reply", "still waiting", "not resolved", "unresolved", "helpless",
    "worst experience", "very bad", "not working", "declined", "pending since",
    "since morning", "multiple times", "again and again", "third time",
    "second time", "escalat",
)
# Without banking-complaint vocabulary the analyser called
# "payment failed 3 times, money debited but not credited" NEUTRAL.
# That single miss is why a domain lexicon beats generic sentiment.

_CUR = r"(?:rs\.?|inr|₹|rupees?)\s*"
_NUM = r"(?:\d{1,3}(?:,\d{2,3})+|\d+)(?:\.\d+)?"   # grouped branch FIRST
# so "Rs 45000" is not cut down to "450"


def extract_amount(text):
    """LARGEST amount mentioned. Auditable, and a wrong amount only shifts
    ONE SLA tier - which is why a regex beats a learned extractor here."""
    low = text.lower()
    found = []
    for m in re.finditer(r"(\d+(?:\.\d+)?)\s*(?:lacs?|lakhs?)", low):
        found.append(float(m.group(1)) * 100_000)
    for m in re.finditer(r"(\d+(?:\.\d+)?)\s*(?:crores?|cr)\b", low):
        found.append(float(m.group(1)) * 10_000_000)
    for m in re.finditer(r"(\d+(?:\.\d+)?)\s*k(?![a-z])", low):
        found.append(float(m.group(1)) * 1_000)
```

```
    for m in re.finditer(_CUR + "(" + _NUM + ")", low):
        try:
            found.append(float(m.group(1).replace(",", "")))
        except ValueError:
            continue
    return max(found) if found else 0.0


def assign_priority(category, text, amount_at_risk=0.0,
                    customer_tier="standard"):
    """Category impact sets the CEILING. Urgency RAISES it, never lowers.

    WHY: a fraud alert is P0 because of WHAT IT IS, not because the customer
    used an angry word. Someone furious about a brochure is still P3.
    """
    from taxonomy import CATEGORY_IMPACT, SLA_RESPONSE_MINUTES
    impact = CATEGORY_IMPACT.get(category, 2)
    base = {4: "P0", 3: "P1", 2: "P2", 1: "P3"}[impact]

    low = text.lower()
    urgency = sum(w for term, w in URGENCY.items() if term in low)
    neg = sum(1 for term in NEGATIVE if term in low)

    source = "signal"
    amount = amount_at_risk
    if amount <= 0:
        amount = extract_amount(text)
        source = "text_extracted" if amount > 0 else "none"

    escalated = None
    if urgency >= 3 and base in ("P2", "P3"):
        escalated = "P1"
    elif urgency >= 1 and base == "P3":
        escalated = "P2"
    if neg >= 2 and base == "P3":
        escalated = escalated or "P2"
    if amount >= 100_000 and base in ("P2", "P3"):
        escalated = "P1"
    if customer_tier == "enterprise" and base == "P3":
        escalated = escalated or "P2"

    final = escalated or base
    return final, SLA_RESPONSE_MINUTES[final], {
        "category": category, "category_impact": impact, "base_priority": base,

        "urgency_score": urgency, "negative_terms": neg,
        "amount_at_risk": amount, "amount_source": source,
        "customer_tier": customer_tier, "final_priority": final,
    }
```

Stage 10 — `src/priority.py`**. The returned reasons dict is not decoration: it is the audit trail, and it is a responsible-AI requirement that ops can actually check.**

```
$ cd ~/drona-practice/src
$ python -c "
$ from priority import assign_priority
$ for cat, txt, amt in [
$     ('payment_failure','my card was declined twice and money was debited', 0),
$     ('payment_failure','my card was declined twice and money was debited', 5000000),
$     ('kyc_verification','I cannot verify my identity, urgent', 0),
$     ('general_enquiry','I am very angry about my brochure, terrible service', 0),
$ ]:
$     pr, sla, why = assign_priority(cat, txt, amt)
$     print(f'{pr}  SLA={sla:>4}m  amt={why["amount_at_risk"]:>10.0f} ({why["amount_source"]})  {cat}')
$ "
```

Run this. Then point at rows 1 and 2.

KEY RESULT

Rows 1 and 2 are the *same text*. With `amount_at_risk=5000000` the priority flips to P0 and `amount_source` flips to `"signal"`. Non-text data outranks prose. And row 4 — an angry customer about a brochure — stays P3, because category impact sets the floor. That contrast is the cleanest 20 seconds in the whole presentation.

Stage 11 — Run everything

One command, one claim.

```
$ cat > ~/drona-practice/RUN_ALL.sh <<'EOF'
$ #!/usr/bin/env bash
$ # WHY a single entrypoint: reproducibility. One command = one claim.
$ set -euo pipefail
$ cd "$(dirname "$0")"
$ export PYTHONPATH=src
$ for s in data exp_oracle exp_fusion exp_conformal; do
$   echo "=== $s ==="
$   python "src/$s.py"
$ done
$ EOF
$ chmod +x ~/drona-practice/RUN_ALL.sh
$ ~/drona-practice/RUN_ALL.sh
```

And the *real* project, for the demo

```
$ cd ~/drona
$ ./install-ml.sh      # venv + torch CPU. The only supported install path.
$ ./run.sh test        # unit tests
$ ./run.sh             # API on :8000, docs at /docs
$ ./run.sh dashboard   # Streamlit on :8501
```

This is your actual deliverable. The practice project exists so you understand *why* each line exists — the demo runs from ~/drona.

NOTE

Be honest about this. The practice scripts are a faithful skeleton, not a byte-for-byte copy of ~/drona. The real project has 23 tests, 8 experiment modules, and artifacts that regenerate byte-for-byte. If the demo matters, verify ~/drona tonight.

Part IV
Defending the work

Eight questions you should expect, with answers that lead with evidence rather than opinion.

Answer *short* first, then stop. If they want depth they will ask.

1. “Why not just use an LLM for this?”

Three reasons, in order.

One: this is a *classification* problem with a closed label set, and LLMs are not the right instrument for closed-set classification when a small fine-tuned model is available.

Two: it has to run at ticket volume on CPU inside a bank — 33 ms and no network is a requirement, not a preference.

Three, and this is the part that should matter: I *measured* it. I built the strong alternative and showed its advantage was noise (p = 0.6061). "Use an LLM" is a decision. This is evidence.

2. “94.6% accuracy — isn't that low?”

For 10-way classification at 18.7:1 imbalance, it's strong. But it's not the number I'd lead with.

The system auto-answers **90.32% of tickets at 100% accuracy** with a statistical guarantee, and escalates the remaining 9.68% to a human. The 94.6% is the *no-gate* figure.

What the business cares about is "how many tickets reach a human wrongly" — and that is measured, at zero.

3. “Your conformal guarantee says 100% accuracy. Suspicious.”

It's bounded above by label noise, and I said so in the report.

Banking77 carries about 14% label noise at the 77-intent level (Ying & Thomas, ACL Insights 2022). P(correct) is capped by disagreement with the *annotator*, not by my model. At the coarse level my label-noise audit found only ~1.1% disputed, so the practical ceiling is far above 0.86.

And at alpha = 0.05 accuracy drops to 99.62% rather than staying pinned at 1.000. Graceful degradation is what you'd expect, and it's a useful sanity check that the guarantee isn't an artefact.

4. “Why is accepted-slice accuracy 100% but overall 94.6%?”

Because abstention removes exactly the hard tail.

The escalated 9.68% are the tickets where the two views disagree — and there the cascade is right only **62.79%** of the time. Those tickets are the ~5-point gap. That's the honest structure of this system: excellent where it's confident, mediocre exactly where it isn't — which is precisely why it escalates there.

5. “What's the weakest part?”

Confidence miscalibration on out-of-distribution input.

“asdkjh qweasd zxcvbn payment failed” returns `payment_failure` at 97.6% confidence. It should be `other`. The threshold can't catch it because the failure is *over*-confidence.

In that case the retriever found nothing good (similarity 0.31) and the confidence came from a lexical anchor on one word. The fix I'd make first is gating on nearest-neighbour *similarity*, not just fused probability — OOD input has no good neighbours, and that's a signal the probability gate cannot see.

6. “Why did you leave the agent advisory-only?”

Because I measured it.

Letting it overrule the cascade made end-to-end accuracy *worse*: 93.17% versus 94.70%. On the disagreement slice the cascade was right 64.34% and the agent only 59.69%.

But the agent's *drafts* were excellent: 100% fact precision, zero invented identifiers. So the right split is machine drafts, human decides. That's a design conclusion from a measurement, not a caution about LLMs.

7. “Your labels were audited and you changed nothing. Why?”

Because both interventions made it worse, and I published that.

Dropping the 101 flagged labels cost 0.15 points. *Relabelling* them cost 2.50. The flagged samples were already agreeing with the annotation 93.9% out-of-fold, so the auditor was identifying the genuinely ambiguous tail rather than clear errors.

Cleaning data on the assumption that flagged means wrong is how you quietly destroy a good decision boundary. I kept the labels and reported the negative result.

8. “What would you do next with more time?”

In priority order, three things.

One: add nearest-neighbour similarity to the gate. That fixes the confidently-wrong OOD case, which is the failure I'd lose sleep over.

Two: SetFit-style contrastive fine-tuning of the small encoder on the training tickets. It's explicitly in the project notes as the next lever, and it targets the *ranking* bottleneck the oracle identified rather than the representation bottleneck.

Three: shadow-mode deployment. Run the cascade beside the human queue for a month and compare escalation quality, not just accuracy.

Part V

One-page summary

Every layer, the decision taken, and the reason for it. If you keep one page, keep this.

Layer	Decision	Why
Labels	77 → 11, with an `other` escape	A forced choice picks wrong *with full confidence*
Split	70/15/15, stratified	18.7:1 imbalance breaks random splits
Features	char_wb(3,5) + word(1,2), union	Typos versus phrases — different failure modes
Retrieval	kNN, `score`**`3`**`, zero learned parameters`	Deterministic, auditable, CPU-cheap
Diagnosis	**oracle@15 = 99.5%, vote = 92.6%**	**Ranking is the bottleneck, not representation**
Model	LinearSVC + kNN, alpha = 0.6	Two independent cheap views beat either alone
Calibration	Temperature, minimising **NLL**	Monotone ⇒ argmax fixed ⇒ use a proper scoring rule
Gate	**Split conformal, alpha = 0.10**	**90.32% auto-answered at 100%, statistically guaranteed**
Priority	Impact ceiling + urgency + non-text	Category sets the floor; data outranks prose
Agent	**Advisory only**	Measured worse: 59.69% vs 64.34% on its own slice
Decision	**Ship the 1.6 MB cascade**	**p = 0.6061 — the transformer's edge is noise**

Table 5.1 — the whole project on one page.

Opening line — memorise verbatim

KEY RESULT

"Banking support tickets go in; a routed, prioritised ticket comes out. Runs offline on CPU — no API keys, no network, no LLM in the serving path. We built two systems and compared them on held-out tickets. DistilBERT won by 0.31 accuracy points. We ran McNemar's test on exactly the tickets where they disagree: it wins 50, loses 44, p = 0.6061. That is a coin flip. So we shipped the other one — a 1.6 MB cascade at 33.3 ms instead of a 268 MB transformer at 105.5 ms. The interesting part isn't that it works. It's which parts we threw away, and why."

Part Appendix

Reference tables

Every figure used in this guide, with its source artifact. No number here was typed from memory.

A. All reported metrics

System	Acc	Macro-F1	ECE	Latency	Size
kNN retrieval only	92.56%	91.75%	0.0785	27.4 ms	—
Linear only	94.09%	93.64%	—	—	—
Cascade (shipped)	**94.60%**	**94.43%**	**0.0170**	**33.3 ms**	**1.6 MB**
DistilBERT fine-tuned	94.91%	94.07%	0.0252	105.5 ms	268 MB

Table A.1 — artifacts 03, 04, 06. DistilBERT: 47.2 min CPU training, 20,431 features, agreement rate 93.43%, accuracy when agree 96.84%, when disagree 62.79%.

B. TF-IDF baselines (artifact 01)

Model	Task	Accuracy	Macro-F1	Macro-P	Macro-R
Logistic regression	coarse 11	92.72%	91.72%	94.40%	89.47%
Logistic regression	fine 77	88.59%	88.51%	—	—
LinearSVC	**coarse 11**	**94.50%**	**94.31%**	**95.08%**	**93.60%**
LinearSVC	fine 77	90.22%	90.00%	—	—

Table A.2 — LinearSVC beat logistic regression by 1.78 points at baseline. That margin is why the cascade uses a margin-based linear model plus an explicit calibrator, rather than a probabilistic linear model out of the box.

C. Retrieval recall@k (artifact 02)

k	coarse char	coarse word	fine char	fine word
1	90.07%	85.84%	80.59%	72.90%
3	96.43%	94.75%	—	—
5	97.76%	96.64%	—	—
10	98.98%	99.13%	—	—
15	**99.54%**	**99.54%**	97.10%	96.59%

Table A.3 — coarsening the labels raised the achievable ceiling. Reporting the coarse number alone would be misleading.

D. Conformal sweep (artifact 08)

alpha	threshold	Promised	Delivered	Acc on accepted	Holds
0.20	0.1265	0.80	0.8202	1.0000	yes
0.15	0.1868	0.85	0.8614	1.0000	yes
0.10	**0.3032**	**0.90**	**0.9032**	**1.0000**	**yes**

alpha	threshold	Promised	Delivered	Acc on accepted	Holds
0.05	0.5758	0.95	0.9475	0.9962	yes

Table A.4 — calibration set: the validation split, held out from training and never the test split. n = 1,963 test tickets.

E. Label-noise audit (artifact 07)

Policy	n_train after	Accuracy	Macro-F1	Delta acc
Baseline	9,158	93.73%	93.00%	—
Drop flagged	9,057	93.58%	93.06%	−0.0015
Relabel flagged	9,158	91.24%	89.82%	**−0.0250**

Table A.5 — 101 of 9,158 coarse labels flagged (1.1%), 5-fold out-of-fold, seed 20261003, OOF agreement with annotation 93.93%. Both clean-up policies hurt.

F. Agent escalation (artifact 05)

Metric	Value
Baseline accuracy (no gate)	94.70%
Disagreement rate	6.57%
Escalated tickets	129
Cascade accuracy on that slice	64.34%
Agent accuracy on that slice	**59.69%**
End-to-end with agent overrule	93.17%
Agent latency	34.99 ms
Mean reasoning-trace steps	11.0
Draft fact precision (mean)	1.000
Drafts with invented identifiers	0

Table A.6 — the agent writes well and decides badly, so it drafts and a human decides.

G. References cited in the project

Source	What it supports
Angelopoulos & Bates, *Conformal Prediction: A Gentle Introduction*, arXiv:1907.07509	The split-conformal construction and its coverage guarantee
Ovadia et al., EMNLP 2024	Retrieval augmented generation competitive with or better than fine-tuning for text classification
arXiv:2403.01432	Retrieval vs fine-tuning trade-offs in production text pipelines
Ying & Thomas, ACL Insights 2022 (insights-1.19)	Banking77 carries ~14% label noise; also the 0.10 cosine silhouette justifying the kNN vote exponent
Tunstall et al., *SetFit*, arXiv:2209.11055	The identified next lever: efficient contrastive fine-tuning
Intercom Fin, Sierra, Decagon — production practice	Human-in-the-loop escalation and abstention as standard practice

Table A.7 — every claim in this guide traces to either a project artifact or a citation.

H. Reproducing every number

```
$ cd ~/drona
$
$ PYTHONPATH=src .venv/bin/python -m drona.experiments.baselines        # 01
$ PYTHONPATH=src .venv/bin/python -m drona.experiments.retrieval_exp    # 02
$ PYTHONPATH=src .venv/bin/python -m drona.experiments.fusion_exp       # 03
$ PYTHONPATH=src .venv/bin/python -m drona.experiments.transformer_exp  # 04  ~47 min CPU
$ PYTHONPATH=src .venv/bin/python -m drona.experiments.escalation_exp   # 05
$ PYTHONPATH=src .venv/bin/python -m drona.experiments.final_comparison # 06
$ PYTHONPATH=src .venv/bin/python -m drona.experiments.label_noise_exp  # 07
$ PYTHONPATH=src .venv/bin/python -m drona.experiments.conformal_exp    # 08
```

Re-running an experiment reproduces its committed artifact *byte-for-byte*. That is the claim to make if anyone asks whether these numbers are real.

I. Glossary

Term	Meaning in this project
Abstention	Refusing to answer and escalating to a human, instead of guessing
Cascade	Two cheap models whose outputs are fused, rather than one big model
Calibration	Making stated confidence match observed frequency
Conformal prediction	A distribution-free method that returns a set or a threshold with a guaranteed error rate
Nonconformity score	1 − confidence, in split conformal
Coverage	Fraction of tickets the system auto-answers rather than escalates
Macro-F1	F1 averaged per class with equal weight — the metric that respects rare classes
ECE	Expected calibration error; average gap between stated confidence and observed accuracy
Discordant pair	A test ticket where exactly one of two systems is correct — the only data McNemar uses
Oracle	An imaginary perfect reranker, used to measure how much information retrieval already has
Stratified split	A split that preserves class proportions in every part
Union TF-IDF	Two vectorisers' matrices concatenated and re-normalised into one feature space